

Machine Learning

Università degli Studi di Verona
Imbriani Paolo, VR553697
paolo.imbriani@studenti.univr.it
Professor Cigdem Beyan

Anno Accademico 2026/2027

Indice

1 Classifiers 3
1.1 Bayes classifiers 4
1.1.1 Problems 6
1.1.2 Solutions (one of many) 6
1.2 MLE in Naive Bayes (discrete case) 7
1.2.1 Continous Features: Guassian MLE 7
1.3 Nonparametric classifiers 7
1.4 Non-parametric techniques 8
1.5 K-Nearest Neighbour 10
1.5.1 Pros and Cons 12
2 Decision Trees 14
2.1 Training Algorithm 15
2.1.1 ID3/Cart Algorithm 16
2.1.2 Entropy 17
2.1.3 Information Gain 17
2.2 Gini Index 19
2.3 Overfitting 19
2.3.1 Summary 19
2.4 Random Forests 20
2.4.1 Ensemble methods 20
2.5 Bagging or bootstrapping 21
2.6 Boosting 21
2.6.1 Stacking 22
3 Linear Discriminant functions 22
3.1 Linear classifiers 24
3.2 Augmented Feature Vector 25
3.3 Gradient Descent 26
3.4 Perceptron 28
3.4.1 Relaxation 28
3.5 MSE 29
3.6 Least MSE 30
3.7 The Ho-Kashyap method 31

1 Classifiers

A fundamental **statistical approach** for classification. Hypothesis:

- The decsiion problem is cast in *probabilistic terms*;
- All relevant **probabilities are known**.

Goal:

- Discriminate the different decision rules using the probabilities and associated costs
- Estimating the **joint probability p(x,y)** from the training dataset.

Example

Let y is what we want to classify and it is to be probabilistically described. There are two classes y_1 and y_2 which correspond to two possible states:

- P(y=y_1) = 0.7
- P(y=y_2) = 0.3

These are called **prior probabilities** and they are known. There are no measuremetns or observations available to help make a decision about which class **y** belong to. How do we decide?

- Decide y_1 if $P(y_1) > P(y_2)$, otherwise decide y_2.
- Given the probabilities (70% for y_1 and 30% for y_2), the decision rule would lead to classifying y as y_1 since 0.7 is greater than 0.3

Example

Having the previous Hypothesis and additionally a single measurement x, which is a random variable that depends on the class y_j, we can get:

$$p(x|y_j)_{j} = 1, 2 = \text{Likelihood}$$

i.e. the probability of having the measurement x given the class y_j. When we fix the measurement x, the higher the likelihood $p(x|y_j)$, the more likely it is that the measurement x belongs to class y_j.

1.1 Bayes classifiers

Bayes formula is a formula that makes us able to compute the posterior probability $p(y_j|x)$, i.e. the probability of having the class y_j given the measurement x.

$$p(y_j \mid x) = P(y_j|x)p(x) = p(x|y_j)P(y_j)$$

that is

$$P(y_j \mid x) = \frac{p(x|y_j)P(y_j)}{p(x)} \iff \text{posterior} = \frac{\text{likelihood} * \text{prior}}{\text{evidence}}$$

The posterior is the probability that y is y_j given the observation x. The most important factor is the product likelihood x prior, the evidence $p(x)$ is simply a scale factor, which ensures that

$$\sum_j P(y_j|x) = 1$$

From the formula of Bayes derives the **Bayes's decision rule**: Decide y_j if $P(y_j|x) > P(y_k|x)$, y_k otherwise.

There's also **Naive Bayes Classifier**, a probabilistic based on **Bayes Theorem**. It assumes *independence* among features. It calculates the posterior probability using:

$$P(y_j|x) = \frac{p(x|y_j)P(y_j)}{p(x)}$$

Assume features $x = (x_1, x_2, ..., x_n)$ are independent given the class label ω:

$$P(x|y) = P(x_1, x_2, ..., x_n \mid y) = \prod_{i=1}^{n} P(x_i|y)$$

The probability of each class is determined from the training dataset.

Example

We want to classify an email as either:

- spam
- not spam

We'll use a few features (words) that appears in the email.

Email	Contains "Free"?	Contains "Win"?	Class
E1	Yes	Yes	Spam
E2	Yes	No	Spam
E3	No	Yes	Spam
E4	No	No	Ham
E5	No	No	Ham

1. Compute class priors:

$$P(\text{spam}) = \frac{3}{5}$$

$$P(\text{Ham}) = \frac{2}{5}$$

2. For each feature (word) and class, we estimate how often the word appears when the email belongs to that class.

$$P(\text{free} \mid \text{spam}) = \frac{2}{3}$$

$$P(\text{free} \mid \text{ham}) = \frac{0}{2}$$

$$P(\text{money} \mid \text{spam}) = \frac{1}{3}$$

$$P(\text{money} \mid \text{ham}) = \frac{0}{2}$$

1. Should be calcuation of evidence but in practice, for classification. $P(x)$ is the same for all classes, it's just a normalization constant. Therefore we can skip its calculation to estimate the posterior probability.

Let's classify a new email: it contains «free», does **not** contain «Win». We compute (ignoring the denominator (evidence) since its the same for both classes):

$$P(\text{free} \mid \text{Free, no win}) \leq P(\text{spam}) \cdot P(\text{free} \mid \text{spam}) \cdot P(\text{money} \mid \text{spam}) = \frac{3}{5} \cdot \frac{2}{3} \cdot \frac{1}{3} = \frac{2}{15}$$

$$P(\text{ham} \mid \text{Free, no win}) \leq P(\text{ham}) \cdot P(\text{free} \mid \text{ham}) \cdot P(\text{money} \mid \text{ham}) = \frac{2}{5} \cdot \frac{0}{2} \cdot \frac{0}{2} = 0$$

Classify an observation x to classify y_j if:

$$P(y_j \mid x) = \frac{p(x|y_j)P(y_j)}{p(x)} \text{ is maximised}$$

- Advanteges:
  - Fast to train and predict, suitable for large datasets,
  - Works well with high dimensional data,
- Limitations:
  - Independecne assumption may not hold in real world scenarios, leading to suboptimal performance,
  - Zero probability problem: requires techniques for unseen features-class combinations.

1.1.1 Problems

To create an **optimal classifier** that uses the Bayesian decision rule, you need to know:

- **Prior probabilites** P(y_i)
- **Class conditional densities** (likelihood) p(x|y_i)

The performance of a classifier *strongly* depends on the **goodness** of these components. But practally all this is information is not available!

More often we only have:

- A vague knowledge of the problem, from which to extract vague **a-priori** probabilities P(y_i)
- Some particularly representative patterns, training data, used to train the classifier (often too few or sparse!)

1.1.2 Solutions (one of many)

We approximate it by **fitting a parametric model** (e.g Gaussian) to the training data and estiamtes its parameters using e.g., **Maximum Likelihood Estimation** (MLE). It's a standard way to estimate the parameters of a probability distribution given a set of observations.

$$\theta_j = (\mu_j, \sigma_j)$$

$$P(x|y_j) \approx N(\mu_j, \sigma_j)$$

MLE chooses the parameters values that make the observed data **the most probable**.

1.2 MLE in Naive Bayes (discrete case)

Suppose you're training a Naive Bayes classifier with categorical features. For each class y you estimate:

- The prior $P(y)$ and the likelihood $P(x|y)$ using MLE.
  1. Prior via MLE:

$$P(y) = \frac{\text{number of samples with class y}}{\text{total number of samples}}$$

  1. Likelihood via MLE:

$$P(x_i = v_k|y) = \frac{\text{count of (x_i = v_k and y)}}{\text{count of (y)}}$$

1.2.1 Continous Features: Guassian MLE

If features are continuous and you assume a Guassian likelihood:

$$P(x_i|y) = \mathcal{N}(x_i, \mu_{iy}, \sigma_{iy}) \longrightarrow P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_{iy}^2}} \exp\left(-\frac{(x_i - \mu_{iy})^2}{2\sigma_{iy}^2}\right)$$

Then MLE gives:

$$\hat{\mu}_{iy} = \frac{1}{N}\sum_{j=1}^{N} x_{ij}$$

$$\hat{\sigma}_{iy}^2 = \frac{1}{N}\sum_{j=1}^{N} (x_{ij} - \hat{\mu}_{iy})^2$$

These are just **sample mean and variance** of that features within each class.

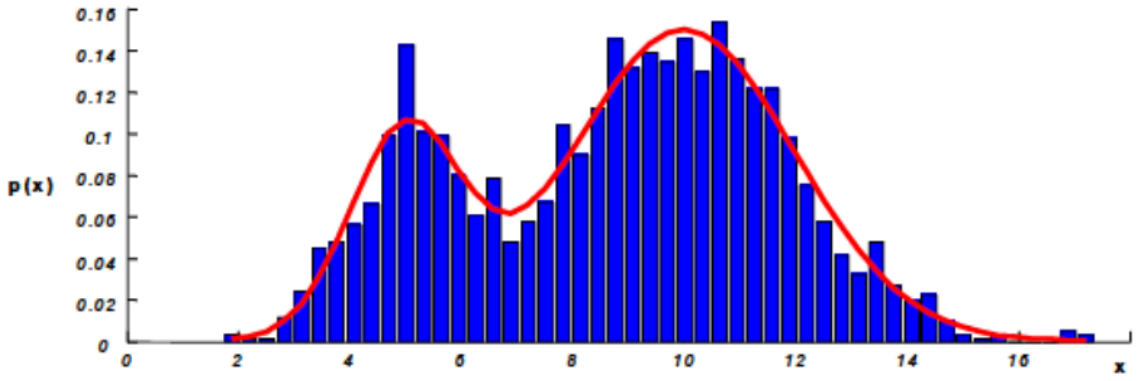
Example

Completa a casa

There's also other solutions for estimating the class conditional densities, such as **non-parametric methods**.

1.3 Nonparametric classifiers

Given a set of examples, model the density function of the data without makig any assumptions about the *form of distribution*. Estimate the probability density function *directly from the samples*. The simplest form of non-parametric density estimation is the **histogram**.



Dividing the sample space into a number of bins and approximate the density at the center of each bin by the fraction of points in the training that fall into that bin.

$$p_{h(x)} = \frac{1}{N}\frac{[\#\text{ of x}\widehat{}\text{(k in the same bin as x)}]}{[\text{bin width}]}$$

The histogram requires two parameters to be defined: bin width and starting position of the first bin. There are few problems with histograms:

- **The density** is not smooth, it **is piecewise constant**, and the estimate is discontinuous at the bin edges.
- The **discontinuities** of the estimate are not due to the underlying density; they are only an artifact of the chosen bin locations
- A much more serious problem is the **curse of dimensionality**, since the number of bins grows exponentially with the number of dimensions.
  - In high dimensions we would require a very large number of examples or else most of the bins would be empty
  - As the number of features or dimensions in a dataset increases, the amount of data required to effectively cover the feature space also increases exponentially.
  - To mitigate the curse of dimensionality, various techniques such as **feature selection**, **dimensionality reduction** (e.g., principal component analysis), and **regularization methods** are employed to reduce the dimensionality of the data while preserving its important characteristics.

1.4 Non-parametric techniques

Before we proceed any futher let us return to the basic defintion of probability to get a solid idea of what we are trying to accomplish:

- The probability that a vector x drawn from a distribution $P(x)$ will fall in a region $\mathcal{R}$ of the sample space is:

  $$P = \int_{x' \in \mathcal{R}} P(x')\ \text{dx'}$$

  where x is a particular vector we are asking about and x' is a dummy variable used to sum/ integrate over all vectors in $\mathcal{R}$.

- Suppose now that N vectors $\{x(1), ..., x(n)\}$ are drawn independently and identically distributed from the distribution. The probability that k of these N vectors fall in $\mathcal{R}$ is given by the binomial distribution:

$$P(k \in \mathcal{R}) = \binom{N}{k} P^k (1-P)^{N-k}$$

  where $\binom{N}{k} = \frac{N!}{k!(N-k)!}$ is the binomial coefficient. and the expected value for k is $E[k] = \text{NP}$ The expectation E is the average value we would get if we repeated drawing samples many times.
- The mean and variance of the ration $\frac{k}{N}$ are

$$E\left[\frac{k}{N}\right] = P \text{ and } \text{Var}\left[\frac{k}{N}\right] = E\left[\left(\frac{k}{N} - P\right)^2\right] = \frac{P(1-P)}{N}$$

  Therefore as N increases, the variance of $\frac{k}{N}$ decreases and the estimate becomes more accurate.

$$P \approx \frac{k}{N}$$

  On the other hand, if we assume that $\mathcal{R}$ is *so small* that $P(x)$ does not vary appreciably within it, then:

$$\int_{\mathcal{R}} P(x')dx' \approx P(x)V$$

  where V is the volume of the region $\mathcal{R}$.
- The general expression for **Nonparametric density estimation** is then:

$$p(x) \approx \frac{k}{NV}$$

  When applying this result to practical density estimation problems, two basic approaches can be adopted:
  - Fix V and determine k from the data. This leads to *kernel density estimation (KDE)*, e.g. Parzen window
  - Fix k and determine V from the data. This given rise to the **k-nearest neighbor** method.

  Both $k - NN$ and KDE converge to the true probability density as $N \to \infty$, provided that V shrinks with N, and that k grows with N appropriately.

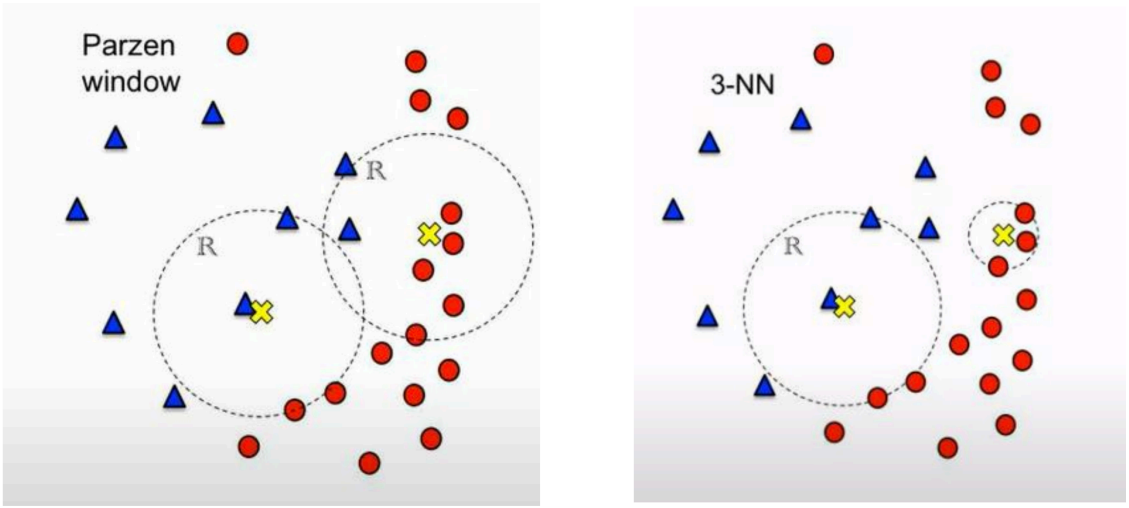


Figura 1: Parzen vs k-NN

1.5 K-Nearest Neighbour

Given an unknown point x^*, we consider $s_i \in \{s_1, ..., s_N\} = S$ the nearest point (NN) of the point x^* if

$$d(x^*, s_i) = \min_l d(x^*, s_l), l = 1, ..., N$$

where $d(.)$ is distance measure defined on the feature space. If you interpret each point of a set S as a prototype of a class, you can then see the equation above as a rule of classification $1 - \mathbb{N}$ associating the sample x to the class j to which the point s_i belongs. To calculate the distance, we usually use the **Euclidean Distance**.

$$d(p, q) = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

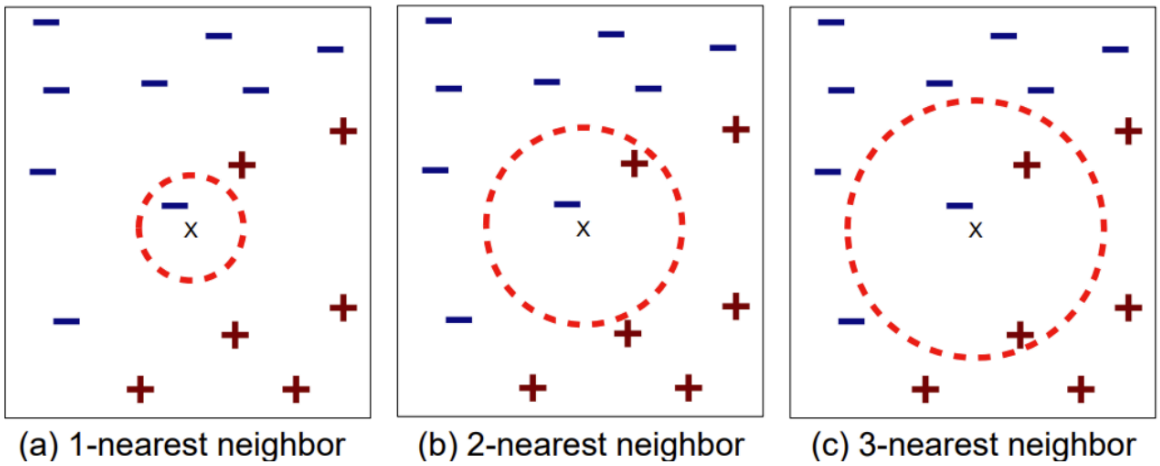


Figura 2: We choose the class using the majority vote of the k nearest neighbors, in (a) and (b) we chose blue class, in (c) we chose red class because of the majority of the neighbours.

All the closest neighbors has the same **weights**, meaning that they are equally imoportant to make a decision. Therefore the algorithm is a very sensitive to the number **k**. This can be by weighing the contribution of neighbors based on distance $w = \frac{1}{d(x^*, s_j)}$, such that the closes neighbour would have more importance.

The coice of k is important because:

- If k is too small, the approach is sensible to noise
- If k is too large, the neighborhood may include points from other classes, leading to misclassification.

To operate correctly the attributes must have the same value scale and must therefore be *normalized* in the preprocessing phase. Normalizing the scale may not b sufficient when there are different data distributions. The mahalanobis distance is a distance measure that takes into account the correlations of the data set and is scale-invariant. It is defined as:

$$D = \sqrt{(x-\mu)^T S^{-1}(x-\mu)}$$

where x is the vector of the observation, μ is the mean vector of the distribution and S is the covariance matrix.

When to use **mahalanobis distance** instead of euclidean distance?

- **Correlated Features**: changes in one features are associated with changes in another
- **Unequal Variances**: MD normalized the distance based on the variances of the features, preventing features with larger variances from dominating the distance calculation.
- **Outlier detection**: Outliers often have a large MD from the rest of the data points, especially if they deviate from the normal covariance pattern.
- In **high dimensional spaces**, ED can be problematic (curse of dimensionality) incorpating information about the covarinance structure, can mitigate this issue to some extent.

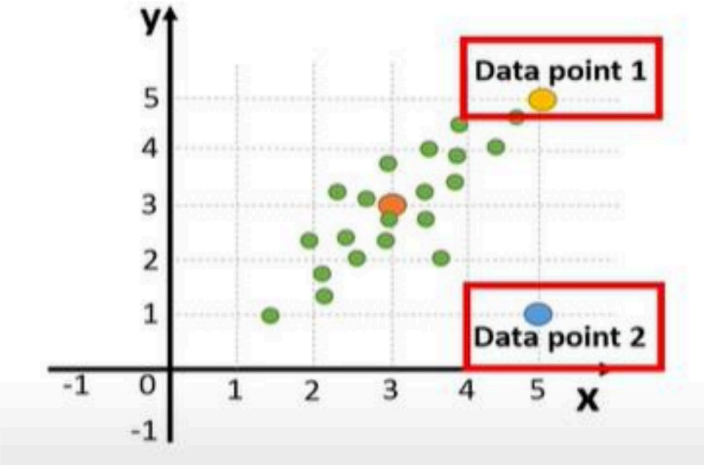


Figura 3: Euclidean distance

$$\text{CENTROID} = \begin{pmatrix} \hat{X} \\ \overline{Y} \end{pmatrix} = \begin{pmatrix} 3.1 \\ 3.0 \end{pmatrix}$$

Euclidean distance between the centroid and the point 1 is:

$$d = \sqrt{(5-3.1)^2 + (3-3.0)^2} = 2.76$$

Euclidean distance between the centroid and data point 2:

$$d = \sqrt{(2-3.1)^2 + (4-3.0)^2} = 2.76$$

These means that both points are equally distant from the centroid, but they are not equally important to the classification.

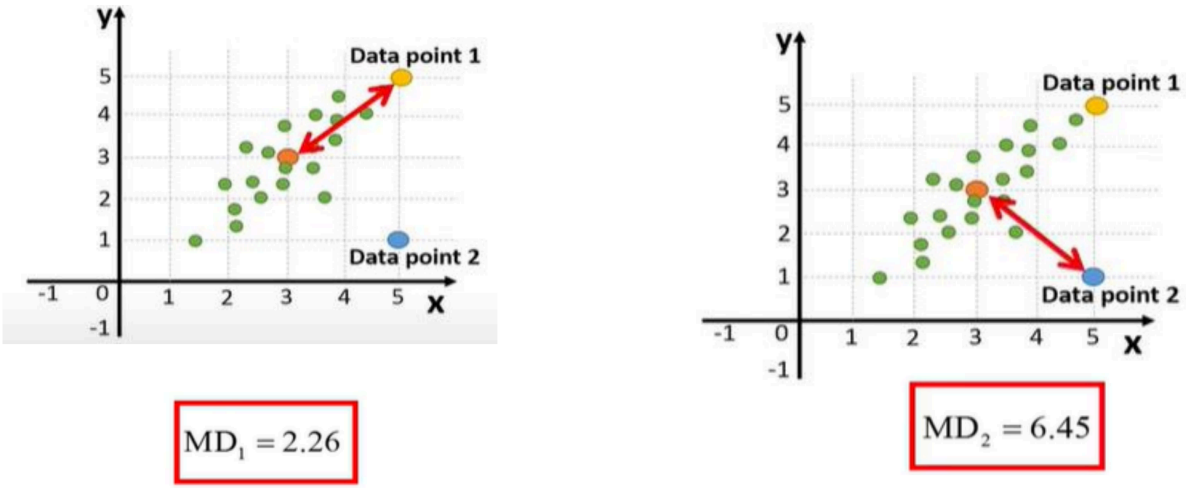


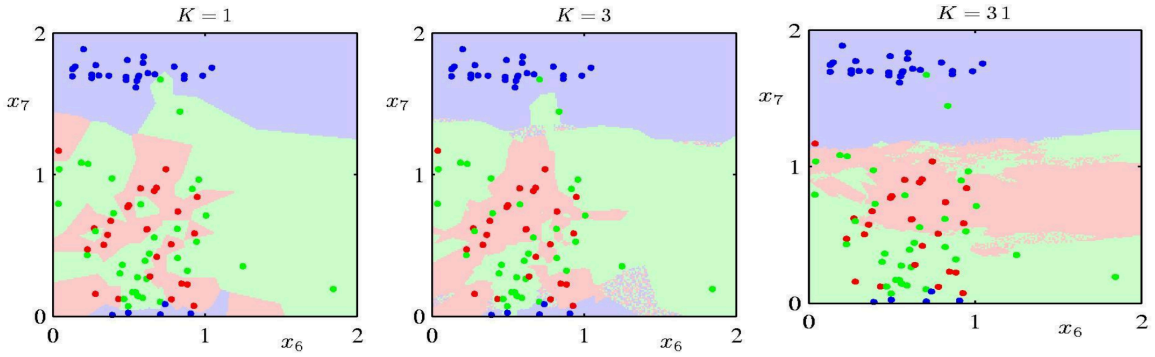
Figura 4: Mahanobis distance

Mahanobis distance takes in account correlation data and since the data is very correlated the data point 1 is more important to the classification than data point 2.

How to select k in $k - \mathbb{N}$ then? Common heuristics are:

- Often 3,5,7
- Choose an odd number to avoid ties

Use your training and validation data to decide k.



What is the role of k?
How does it relate to overfitting and underfitting?

Figura 5: We can see how the accuracy changes with different values of k. The best value of k is the one that gives the highest accuracy on the validation set. And with the K you can either overfit or underfit

If there are undetermined cases (ties) there are some alternatives:

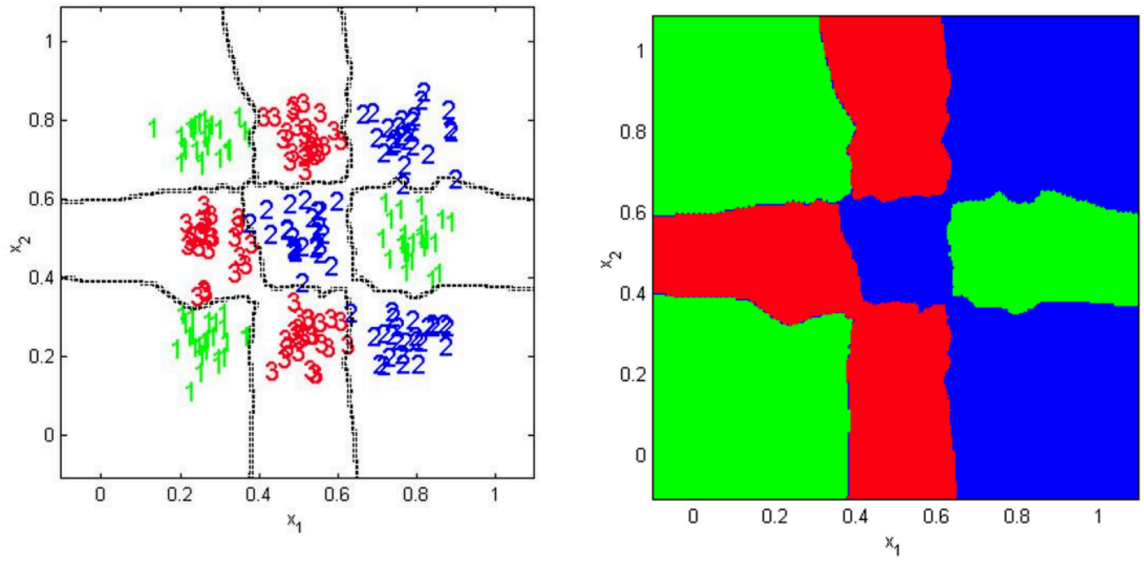
- Arbitrary choice
- Assign x to the class that has the nearest average sample
- Assign x to the class that has the k_i samples with the least distance from it

1.5.1 Pros and Cons

The pros of K-NN are:

- Almost no assumptions about the data
- Non-parametric
- There is no training phase

- Allows us to construct non-linear class boundaries and are therefore more flexible.



The cons of K-NN are:

- Need to handle missing data,
- Need to be aware to overfit or underfit with the K and also the distance metric must be chosen carefully,
- Sensitive to class outliers and many irrelevant attributes,
- Computationally expensive:
  - Poor scalability, we need to perform as many comparisons as labaled data.
  - You need to store all data points, so it's better when the number of samples N is low.

Going back to PDF (probability density function) estimation:

- Instead of fixing V as in case for kernels, we fix the value of k and enlarge

the radius of the sphere to include k samples.

- Let's see the priori probabilty:
  - Given a class y_j, the frequency of occurence of samples N_j of class j

  is generally simply measured in relation to the total number of samples N, i.e:

$$P(y_j) \equiv \hat{p}(y_j) = \frac{N_j}{N}$$

  - 2 classes y_i and y_j containing N_i and N_j samples, respectively. $N = N_i + N_j$.
  - The local density estimation for y_i is calculated as (and for similarly for y_j)

$$\hat{p}(x|y_j) = \frac{1}{V}\frac{k_i}{N_i}$$

  such that the ratio of $k_i(k_j)$ points to the total $N_i(N_j)$ belonging to class $y_i(y_j)$, and the volume V: The Bayes rule says $p(x|y_j)P(y_i) > p(x|y_j)P(y_j)$ then

$$\hat{p}(x|y_i)P(y_i) > \hat{p}(x|y_j)P(y_j) \longrightarrow \frac{1}{V}\frac{k_i}{N_i} \cdot \frac{N_i}{N} > \frac{1}{V}\frac{k_j}{N_j} \cdot \frac{N_j}{N}$$
$$\longrightarrow k_i > k_j$$

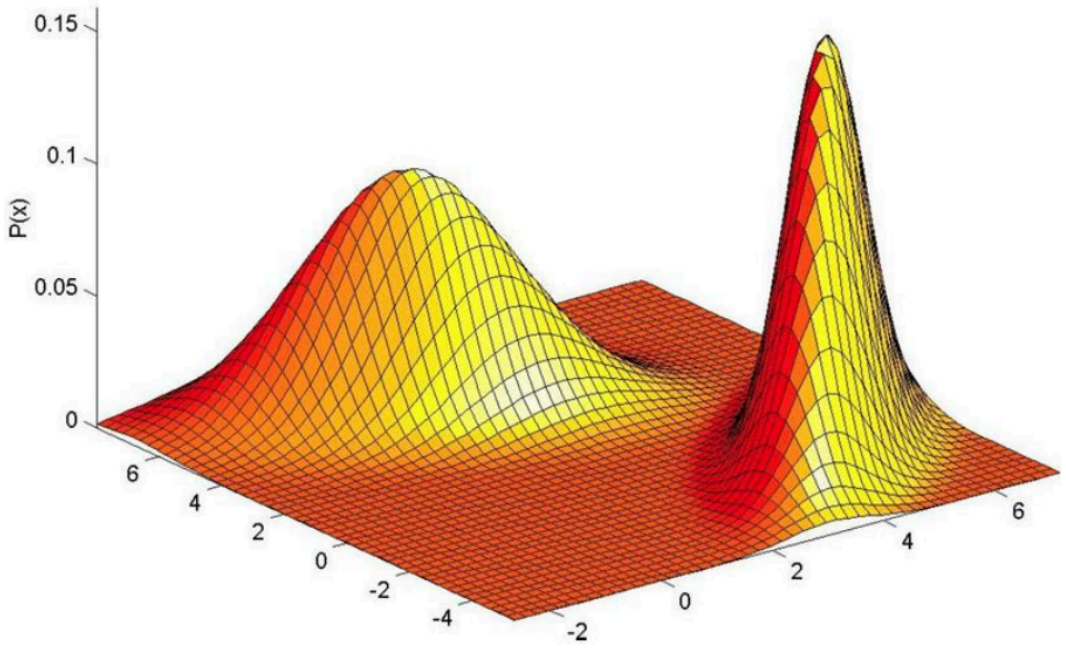


Figura 7: The true density, a mixture of bivariate Gaussians

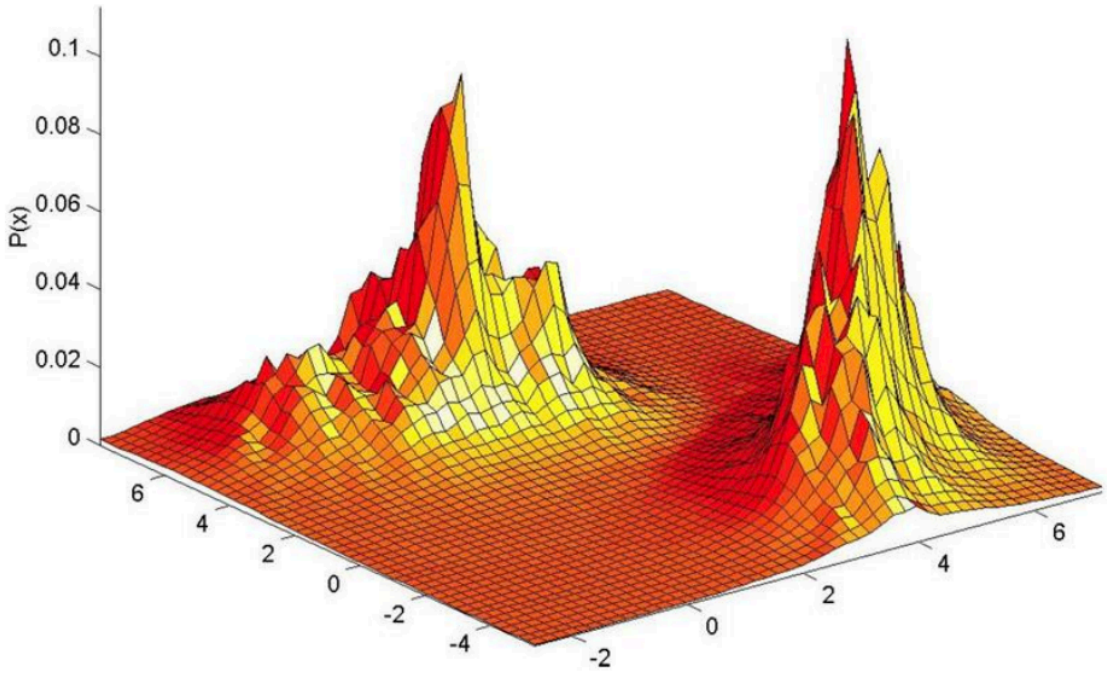


Figura 8: Density estimate for k = 10 neighbous and N = 200 examples

Notice that the estimate can be «jagged» and discontinuities in the slopes generally occur along lines away from the positions of the points themselves.

2 Decision Trees

We are going to build a tree using the training data. It is composed of terminal (or leafs) nodes and non-terminal nodes.

- **Non terminal nodes** have 2+ children and implement a routing function.
- **Leaf nodes** have no childres (terminal) and implement a prediciton function.

There are no cycles, all nodes have at most 1 parent excepting one, **the root node.**

A decision tree takes an input $x \in X$ and routes it through its node until it reaches a leaf node. Each nde represents a variable (feature). An adge to achild node represetns the predicted value for the class based on the values. In the leaf, a prediction takes place.

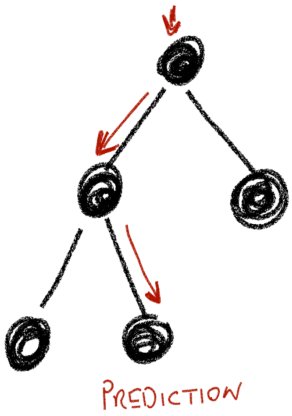


Figura 9: Decision Tree

Each non-terminal node $\text{Node}(\varphi, t_L, t_R)$ holds a routing function $\varphi \in \{L, R\}^{\mathcal{X}}$ such that there exists the left child t_L and the right child t_R. When X reaches the node, it will go to the left child if $\varphi(X) = L$ and to the right child if $\varphi(X) = R$. Here we assume that the tree is binary, but it can be generalized to more than 2 children. Each node $\text{Leaf}(h)$ holds a prediction function $h \in \mathcal{Y}^{\mathcal{X}}$ such that when X reaches the leaf, it will output the prediction $h(X)$. The decision tree function:

$$f(x)_t = \begin{cases} h(X) \text{ if } t = \text{Leaf}(h) \\ f(x)_{t_{\varphi(x)}} \text{ if } t = \text{Node}(\varphi, t_L, t_R) \end{cases}$$

2.1 Training Algorithm

Given a training set: $\mathcal{D} = \{z_1, ..., z_n\}$ we want to find a tree $t^\star$:

$$t^\star = \arg \underbrace{\min}_{t \in T} E(f_t; D_n)$$

We want to find the tree that minimises the error $E(f_t; D_n)$ on the training set. This optimisation problem has many solutions. Thus we need to impose constraints, e.g., most compact tree otherwise it could be **NP-Hard**.

You can also use *continous attributes*. Pick threshold to create a split.

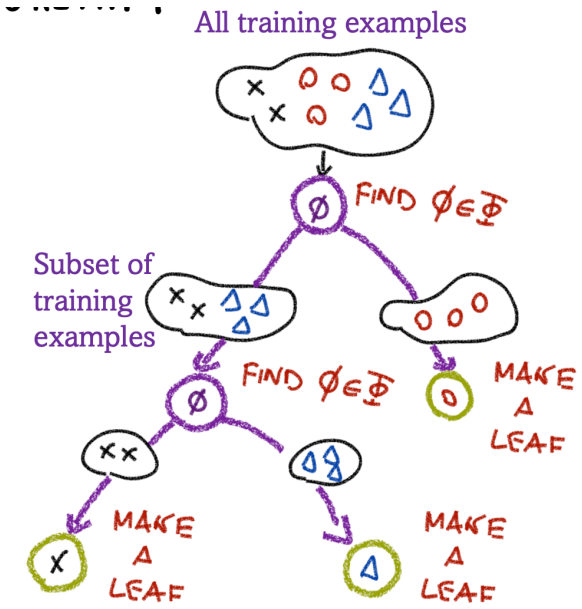


Figura 10: Decision Tree

- We need to fix a set of leaf predictions
- Fix a set of possible splitting functions
- Tree-growing strategy recursively partitions the training set

and decides whether to grow the leaves or non terminal nodes.

2.1.1 ID3/Cart Algorithm

To explain this algorithm let's use an example. Let's look at this problem.

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

[See: Tom M. Mitchell, *Machine Learning*, McGraw-Hill, 1997]

Figura 11: Decision Tree

- 14 training samples
- 4 features: Outlook, Temperature, Humidity, Wind
- 9 of them class «Yes» and 5 of them class «No»



How do you pick the best attribute?

- Which split do you find better and why? **Outlook**, because it has a pure subset. A «pure set» is an option that there will no way that the «no» class will be picked.
- If you look at «wind», which subset of wind is better than the other? **Weak is better**, even though is not pure.

We want to measure **purity** of the split.

- We want to have more certain about Yes/No after the split.
- Pure set $\longrightarrow$ completely certain about Yes/No.
- Impure set $\longrightarrow$ completely less certain.

This measure should be symmetric, i.e., it should not matter which class is Yes or No. There are a lot of diffrent ways to measure the purity, for example **entropy**.

2.1.2 Entropy

Assume that we have binary classes (positive and negatives). For each class:

- p(+) is the proportion of positive in a subset
- p(-) is the proportion of negative in a subset

An **entropy** of *a subset*:

$$H(S) = -p(+)\log_2 p(+) - p(-)\log_2 p(-)$$

or

$$H(S) = \sum_{c \in C} -p(c)\log_2 p(c)$$

- Impure set $\longrightarrow$ high entropy

$$H(S) = ... = 1$$

- Pure set $\longrightarrow$ low entropy

$$H(S) = ... = 0$$

The higher the entropy the more impure the set is. The lower the entropy the more pure the set is.

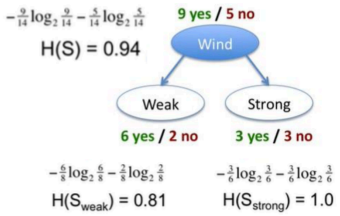
2.1.3 Information Gain

We want to have maby items in the pure sets. We calculate the expected drop in entropy after each split:

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{ Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

where:

- V is possible values of A,
- S the set of examples,
- S_v subset where $X_A = V$.



Mutual information between A and class labels of S:

Gain (S, Wind)
= H(S) – $^{8}/_{14}$ H(S_{weak}) – $^{6}/_{14}$ H(S_{weak})
= 0.94 – $^{8}/_{14}$ * 0.81 – $^{6}/_{14}$ * 1.0
= 0.049

We use information gain to decide which attribute to pick. We want to maximize the «information gain».

- Take every attribute that you have in your data
- Compute gain for the attribute
- Select the attribute with the highest gain
- Highest? The one that reduces the uncertainty the most.

We consider one split at a time, but remember the procedure is recursive. The pros are that:

- Decision trees are interpretable,
- It is possible to read the rules of the tree. There is concise

description of what makes an item positive/negative.

- No «black box», very important for users.

The problems of Information Gain is that sometimes just having pure subsets is not enough. It generalizes poorly in testing data. How to handle this? One possibility is to use the heuristic **Gain Ratio**:

$$\text{GainRatio}(S, A) = \frac{\text{Gain}(S, A)}{\text{SplitEntropy}(S, A)}$$

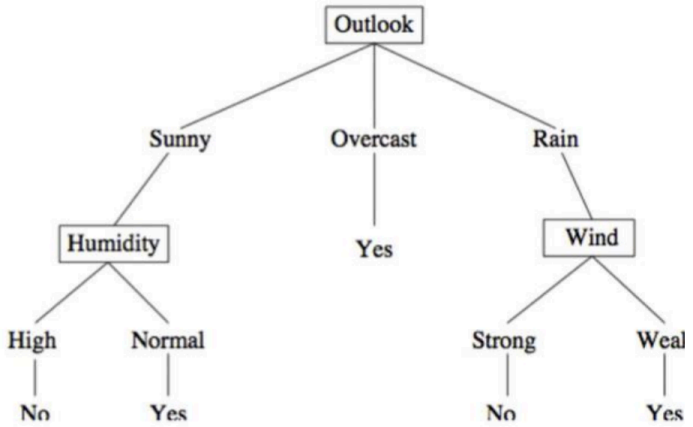
$$\text{SplitEntropy}(S, A) = -\sum_{v \in \text{ Values}(A)} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}$$

where:

- A is the attribute,
- V are the possible values of A,
- S is the set of examples,
- S_v is the subset where $X_A = V$,

It *penalizes attributes with many values.*

Rule: (Outlook = Overcast) ∨
(Outlook = Rain ∧ Wind = Weak) ∨
(Outlook = Sunny ∧ Humidity = Normal)



What are other measures?

- Entropy for classifications tasks, maximize the **information gain**
- Gini Impurity for classifications tasks, where you also need to maximize to **information gain**
- Variance, for regression tasks, where you need to minimize the **variance**.

Another problem with Decision trees is that they are very greedy so they may not find the best tree. There are also exponentially many trees. Also only axis-aligned splits are considered.

2.2 Gini Index

The original CART algorithm uses the GINI impurity, whereas ID3 uses Entropy or Information Gain.

$$\text{Gini} = 1 - \sum_{i=1}^{n} p_i^2$$

While buildiing the decision tree, we would prefer to choose the features with the *least Gini index* as the root node. This method is non-paramethric model since we are not assuming the distribution by calculating mean or variance.

2.3 Overfitting

Decision trees are non-parametric models with a structure that is determined by the data. As a result, they are flexible and can easily fit the training set, with a high risk of overfitting. Standard techniques to improve generalisation apply also to decision trees (early stopping, regularization, data augmentation, complexity reduction, **ensembling**). A technique to reduce complexity a posteriori is called **pruning** (an operation that is applied after tree building).

2.3.1 Summary

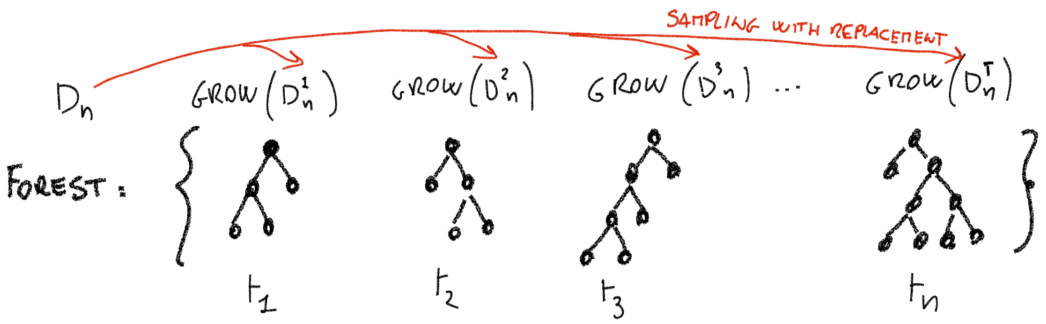
ID3 grows decision tree from the root to down:

- Greedily selects next best attribute using Information Gain,
- Entropy: How uncertain we are in terms of Yes/No in a set
- Inf. Gain: reduction in uncertainty following a split
- Searches a complete hypothesis space
- Prefers smaller trees, high gain at the root

- Overfitting addressed by post-pruning -Prune nodes, while accuracy increases on validation set
- Fast, compact, interpretable

2.4 Random Forests

Random forest are *ensembles* of decision trees. Each tree is typically trained on a bootstrapped version of the training set (sampled with replacement).



To improve the performance, it's better to not put any bias inside the training set, so you will need to balance the tree.

- Split functions are optimised on randomly sampled features or are sampled completely at random.
- This helps to obtain decorrelated decision trees
- The final prediction of the forest is obtained by averaging the

predictions of each tree in the ensemble $\mathcal{Q} = \{t_1, ..., t_n\}$.

$$f_{\mathcal{Q}}(x) = \frac{1}{T} \sum_{j=1}^{T} f_{t(x)}$$

where $\frac{1}{T}$ is the average of T number of trees.

2.4.1 Ensemble methods

The concept of combining multiple models to improve performance is called **ensemble learning**. There are different types:

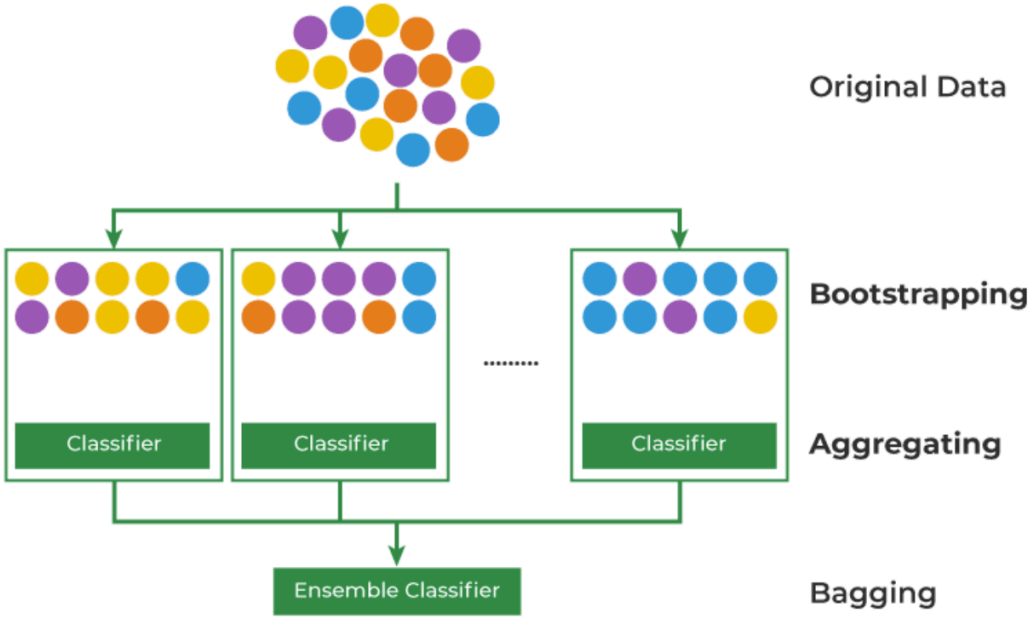
- Bagging
- Boosting
- Stacking

Many times, ensemble methods are made with «weak learners», rather than having a few complex methods. A **weak learner** is a classification model that performs slightly better than random guessing.

- For instance a binary classifcation task with balanced classes, this can be a classifier performing slightly better than 50% accuracy. For example k-NN with k=1 operating on one or a subset of input variables. Or a decision tree with a single node operating on one feature.

2.5 Bagging or bootstrapping

Bagging is a technique in which we train N models on N different sampling of our dataset. This way, all models will have different predictions; we will average them in the end to obtain a single value.

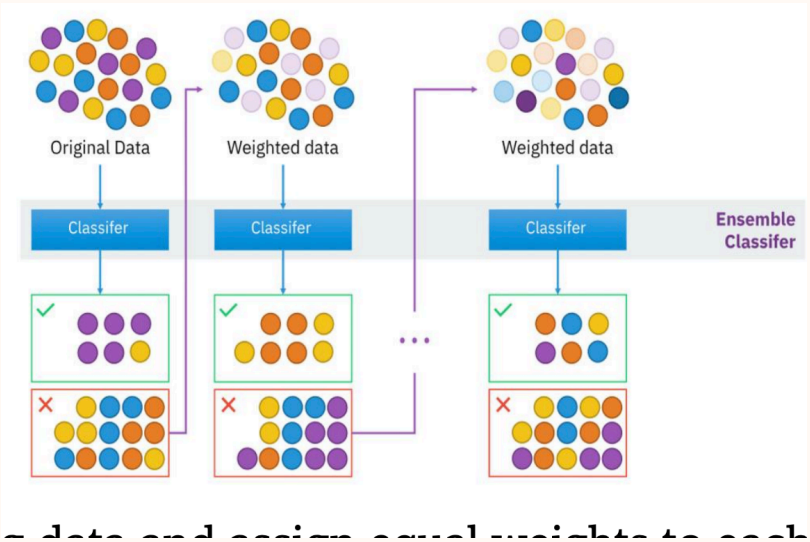


2.6 Boosting

Trained models are weighted based on their performance. The weak classifiers are learned sequentially. The more the previous classifier makes mistakes on a dataset, the more they are weighted for the enxt one.

Example

One example is called **AdaBoost** (Adaptive Boosting). It works by weighting the instances in the training dataset based on the accuracy of previous classifications.

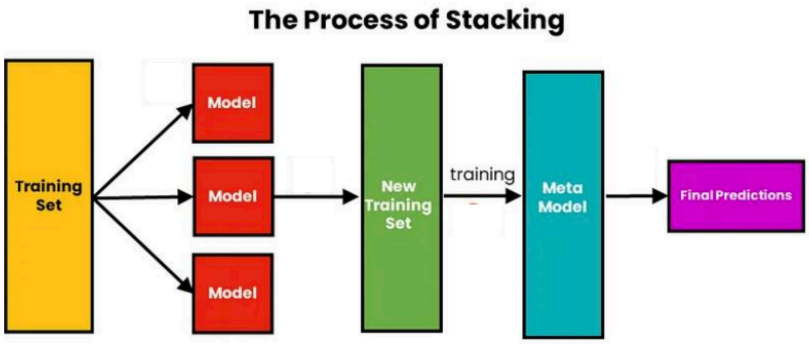


1. Use all training data and assing equal weights to each training sample to build a weak classifier. Check which training samples are correctly classified and which are not.
2. Assign more withgts to the incorrectly classified samples such thtat classfying them correctly becomes more important.
3. Re-iterate step 2 until all data points are correctly classified or until a maximum number of iterations is reached.

2.6.1 Stacking

Combines the predictions of multiple ML models (*called base models*) to improve the overall performance. Different from boosting and bagging, the used base models are heterogenous. Can be used for both **regression** and **classification** problems. A powerful ensemble learning algorithm that can often improve the performance of individual models.

The basic idea of stacking is to first train a set of base modesl on the same dataset. The predictions of these base models are then used to train a meta-model, which is a higher-vel model that learns *how to combine the prediciton of the base models* to make a better prediction.



3 Linear Discriminant functions

The objective of this topic is to presetn methos for learning linear discriminant functions of the form:

$$g(x) = w^T x + w_0 \iff \begin{cases} g(x) > 0 & x \in \omega_1 \\ g(x) < 0 & x \in \omega_2 \end{cases}$$

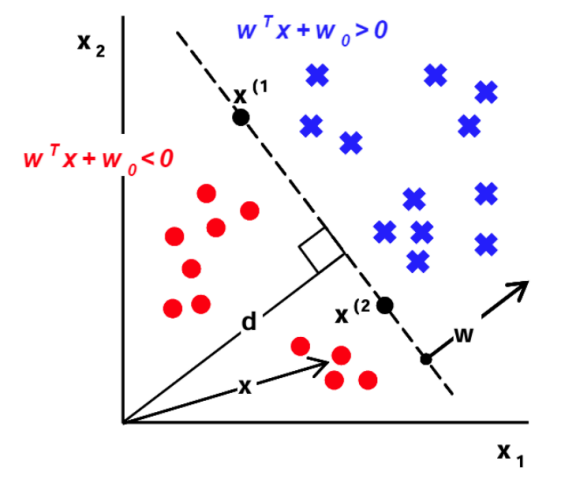
where w is the weight vector and w_0 is the bias term.

$$x = [x_1, x_2, ..., x_n]$$
$$w = [w_1, w_2, ..., w_n]$$

For a problem with 2 classes the goal is to create a linear function $g(x)$ that separates the example belonging to the two classes.

$$g(x) = 0 \rightarrow \text{either class}$$

Example



Suppose we have se tof m samples $x_1, ..., x_m$ some tagged ω_1 and others tagged ω_2.

- We want to use these samples to determine weights w and w_0

of the linear discriminant function.

- Suppose also that there is a solution for which the probability of error is very low.

Thus, a reasonable approach is to look for a weight vector that classifies all of the samples correctly or that the probability of making a mistake is zero. If such a weight vector exists, the samples are said to be **linearly separable**.

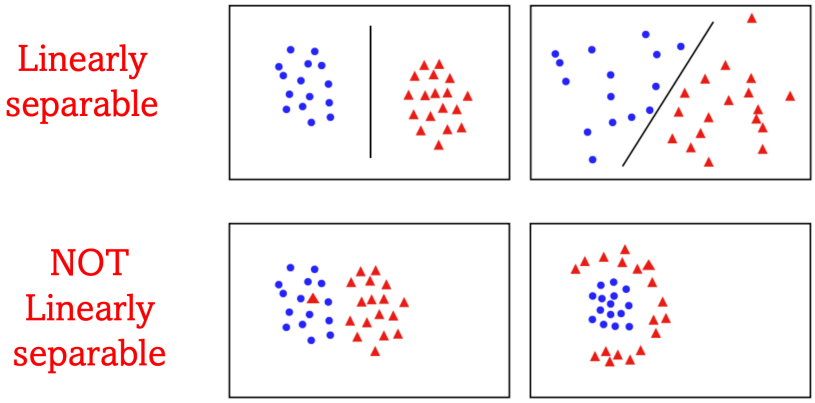


Figura 21: Linear separable data

Linear discriminant functions are optimal for Gaussian distributions with **equal covariance**. May not be optimal for other data disrtributions, but they are very simple to use. Knowledge of class identies is not required when using linear discriminant functions. Therefore we can say that this is a **non-parametric** method.

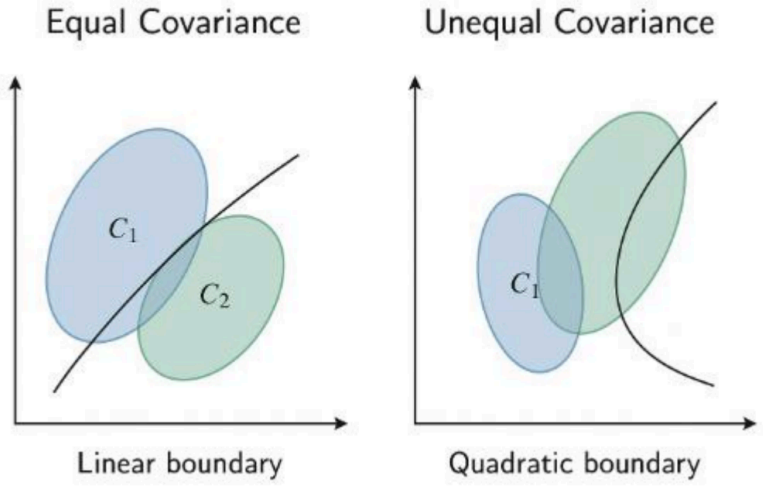


Figura 22: Linear separable data

3.1 Linear classifiers

Every linear discriminant function defines a linear classifier, but not every linear classifier is necessarily described in the form of a «discriminant function».

Decision boundary: $g(x) = w^T x + w_0 = 0$;

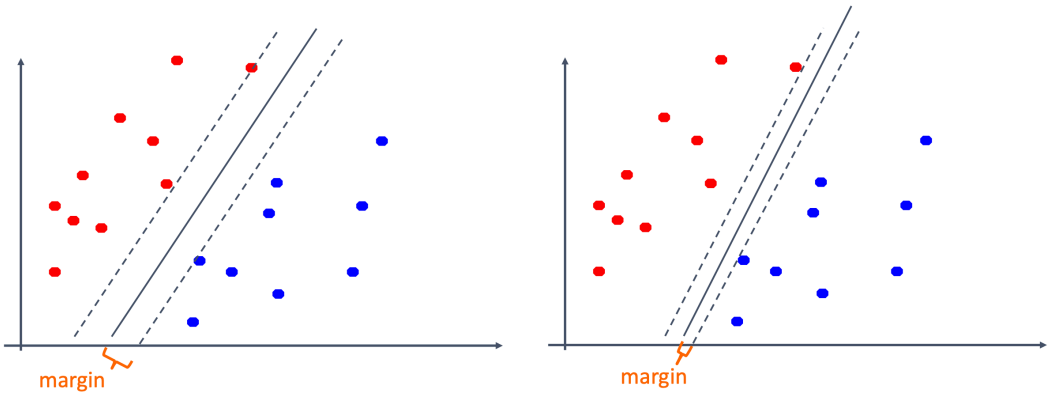
- In 2D, is a line
- In 3D, is a plane
- In nD, is a hyperplane.

The training data is used to learn $\mathbf{w}$. *Only w is needed to classify the new data points.* If a solution vector exist (if there's an hyperplane) then: Which hyperplane a lienear classifier choose when the data is linearly separable? *The solution vector is not unique.*

One possibility is to seek a **unit-length weight vector** that maximizes the minimum distance from the samples to the separating plane. Another possibility is to seek the *minimum-length weight vector* satisfying:

$$w^t x_i \geq b, \forall i$$

where b is a positive constant called the margin. These techniques attempt to find a solution vector closer to the "middle" of the solution region: in this way, the resulting solution is more likely to classify new test samples correctly.



The **margin** of a classifier is the distance to the closest points of either class.

In case of a multi-class problem:

- One-vs-rest: C-1 Classifiers are constructed, one for each class C_i versus non-C_i.
- One-vs-one: C(C-1)/2 Classifiers and then classify a test sample following a majority voting.

Both lead to areas of **ambiguity**.

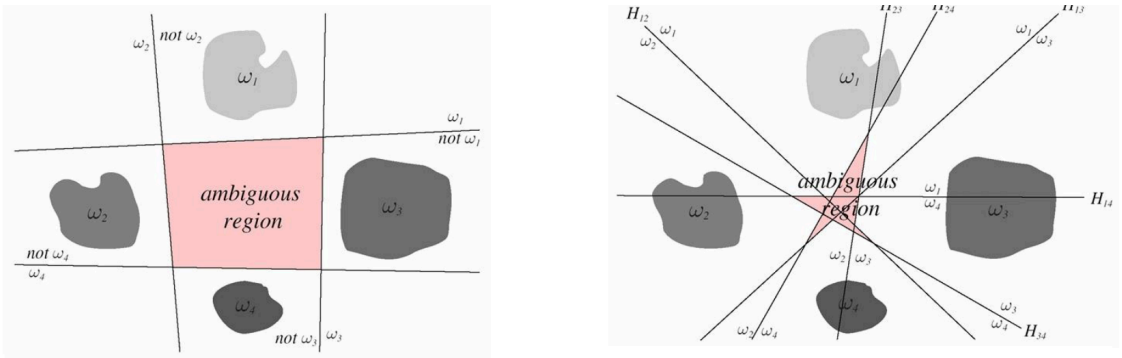


Figura 24: Multi-class classification: One-vs-rest and One-vs-one

3.2 Augmented Feature Vector

We can write the linear discriminant function as:

$$g(x) = (w_0 \;\; w)^T \begin{pmatrix} 1 \\ x \end{pmatrix} = a^t y = g(y)$$

where y is called the augmented feature vector. We added a dummy dimension to get a completely equivalent new homogeneneous problem. Feature augmentation is done only for simpler notation. Throughout these notes we replace a with w for simplicity, since they both refer to *weights*. The old problem:

$$g(x) = w^T x + w_0$$

The new problem:

$$g(y) = a^t y$$

For the rest of this section, assume we have two classes.

Training error is 0 if:

$$\begin{cases} g(y_i) > 0 & \forall y_i \in c_1 \\ g(y_i) < 0 & \forall y_i \in c_2 \end{cases}$$

$$\begin{cases} a^t y_i > 0 & \forall y_i \in c_1 \\ a^t y_i < 0 & \forall y_i \in c_2 \end{cases}$$

$$\begin{cases} a^t y_i > 0 & \forall y_i \in c_1 \\ a^t (-y_i) > 0 & \forall y_i \in c_2 \end{cases}$$

This suggest a so called «normalization»:

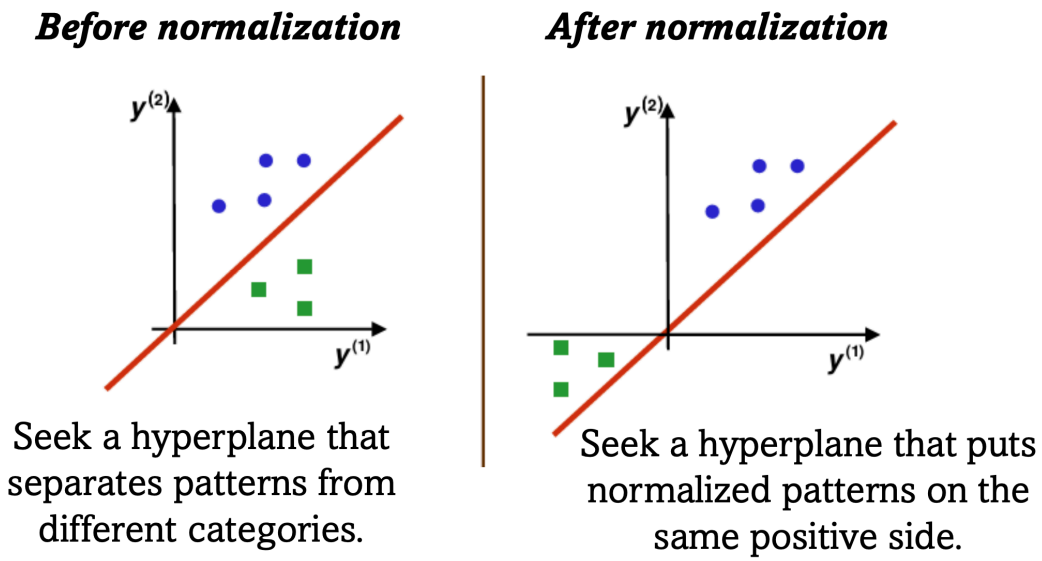
- Replace all examples from class 2 by their negatives:

$$y \to -y_i, \forall y_i \in c_2$$

- See weight vector a such that

$$a^t y_i > 0, \forall y_i$$

If such a exists, it is called a separating or *solution vector*. Original samples $x_1, ..., x_n$ can indeed be separated by a line in that way.

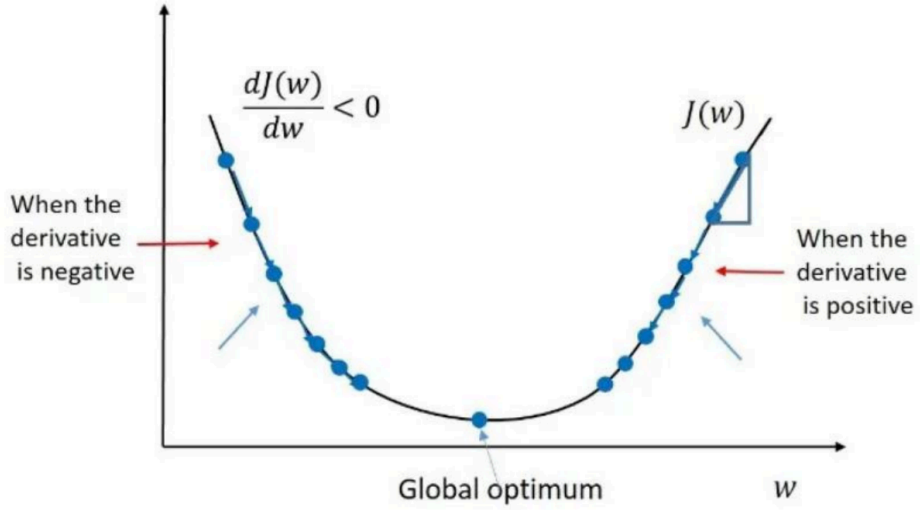


Coming back to the “which hyperplace” question we can see that many solutions for the weight vector a exist.

The solution region corresponding to a consists of all points classified on one side of the hyperplane, where the hyperplane is perpendicular to the normal vector **a**.

3.3 Gradient Descent

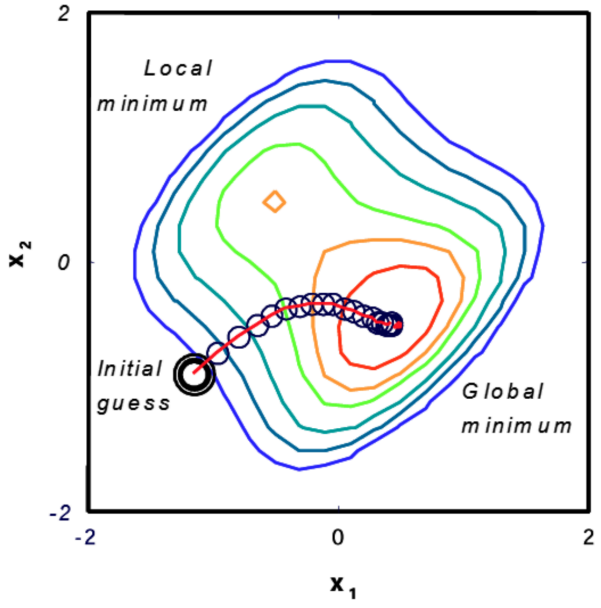
Gradient Descent is one of the simples approaches to calculating a *discriminant function*. It is an iterative method for progressively adjusting the **weights**. The gradient vector in space W points to the direction on a **maximum deviation** of a function to be maximized/minimized.



The procedure consists of updating the vlaue of the weight vector at the $k+1$ step with a contribution proportional to the gradient module calculated at the previous step:

$$w(k+1) = w(k) - \rho_k \nabla J(w(k))|_{w=w(k)}$$

where $J(w)$ is an **evaluation function (loss function)** that must be minimized. $J(w)$ is chosen in such a way as to reach the minimum when w approaches the optimal solution; That is, J(w) should be convex, and w is the solution vector. The minimum of J(w) is obtained by moving w in the direction opposite to the gradient (in the direction of steepest descent). Gradient descent is **guaranteed to find only a local minimum**. Very popular, as it is simple and applicable to any function.



∇ is the gradient operator symbol given by:

$$\nabla J = \begin{pmatrix} \frac{\partial J}{\partial w_1} \\ \frac{\partial J}{\partial w_2} \\ \dots \\ \frac{\partial J}{\partial w_n} \end{pmatrix} \qquad w(k+1) = w(k) - \rho_k \nabla J(w(k))|_{w=w(k)}$$

ρ_k (learning rate) is an appropriate scalar value that varies with the iteration k, fixing the «magnitude» of the correction. At the k-th iteration:

- Start with a chosen weight vector $w(1)$ and calculate its gradient $J(w(1))$
- Then move from $w(1)$ in the direction of the steepest descent (opposite to the gradient) by a fixed step size ρ_k to obtain a new vector $w(2)$.

The main issue is how we use the optimal choice of ρ_k.

- if ρ_k is too **large**, gradient descent may overshoot the minimum and possibly never find the minimum.
- if ρ_k is too **small**, gradient descent may take a long time to converge to the minimum.

3.4 Perceptron

Let's learn a **binary classifier** with linear discriminant function. To do so we need to choose a $J(w)$ such that the gradient descent can solve it, in this way we can find the minimum of $J(w)$. A good choice is **Perceptron** criterion function:

$$J(w) = -\sum_{x_i \in M} y_i w^T x_i$$

where X si a set of samples misclassified by w. J(w) is a non-negative since $w^t x_i < 0$ for all misclassified samples. Do not forget to «normalize» the training set by replacing all examples of ω_2 by their negatives. We want to use gradient descent to solve the $J(w)$ so that it can converge, however, for such data (not linearly separable), *we cannot converge*. To find the minimum of $J(w)$, we use gradient descent, which is defined by:

- i-th component of the gradient vector is equal to:

$$J(w) = -\sum_{i \in X} w^T x_i \Rightarrow \nabla J = -\sum_{i \in X} x_i$$

$$w_{k+1} = w_k + \rho_k \sum_{i \in X} x_i \text{ when } \rho_k = \frac{1}{k}$$

Notice that ∇J does not depend on w but only on the misclassified samples. Perceptron is the first **neural network**, whcih was then evolved to more complex versions such as **multi-layer perceptron**.

Example

3.4.1 Relaxation

There are different relaxation methods:

- The $J(w)$ takes the form:

$$J_{q(w)} = \sum_{x_i \in M} \left(w^T x_i\right)^2$$

  also this function takes into account the missclassified samples and is minimised when **w** is the solution vector. The main difference is that its **gradient is now continuous** whereas

the gradeint of the perceptron function J is not. Such functions are suitable for finding smooth surfaces but this may lead problems.

- Another option for $J(w)$ is:

$$J(w) = \frac{1}{2} \sum_{x_i \in M} \frac{\left(w^T x_i\right)^2}{\|x_i\|^2}$$

3.5 MSE

The classical MSE criterion provieds an alternative to the perceptron rule. The perceptron rule only consider misclassified samples, since these are the only ones that violate the $J(w)$. Instead, the MSE criterion looks for a solution involving **all of the samples**. The goal is to verify $w^t x_i = b$ where the b are some arbitrarily specified positive constants. We have now changed the problems:

- **Perceptron**: $w^t x_i > 0$ for all samples x_i this solve sthe system of linear inequalities
- **MSE**: $w^t x_i = b$ for all samples x_i this solve the system of linear equations.

The MSE procedure is the following:

- Choose positive constants: $b_1, ..., b_n$
- Try to find a weight vector **w** such that $w^t x_i = b$ for all samples x_i.
- If we can find a weight vector w then w is a solution because b_i's are all positive.
- Consider all the samples.

For consistency, we will contiue assuming that examples from class ω_2 have been replaced by ther negative vector, although this is not a requirement for the MSE solution. Thus we have:

$$X_{i,n+1} = 1 \text{ for } \omega_1 \text{ and } X_{i,n+1} = -1 \text{ for } \omega_2$$

We then obtain N equations with N = N_1 + N_2. We need to salve N equations. In matrix notation we have:

$$X \cdot w = b$$

when:

$$X = \begin{pmatrix} x_{1,1} & ... & x_{1,n+1} & \pm 1 \\ ... & \ddots & & \vdots \\ x_{N,1} & ... & x_{N,n+1} & \pm 1 \end{pmatrix}$$

$$b = \begin{pmatrix} b_1 \\ ... \\ b_N \end{pmatrix}$$

$$w = \begin{pmatrix} w_1 \\ ... \\ w_{n+1} \end{pmatrix}$$

Thus we need to solve a linear system:

$$X \cdot w = b$$

Generally $N \gg n + 1$ so there are more equations than unknowns.

$$X \cdot w = b \rightarrow w = X^{-}1b$$

let's now inject the error vector (needed for the loss function) defined as

$$e = X \cdot w - b$$

Imagine this as b is the real class label and Xw is your predictions. MSE finds w, which minimises the length of the error vector e. Thus minimise the minimum squared error criterion function $J(w)$:

$$J_{M(w)} = (Xw - b)^t(Xw - b) = \|Xw - b\|^2$$

Unlike the perceptron criterion, by the MSE function is quadratic and differentiable. Therefore we can find its minimum analalytically by setting the gradient to zero. The gradient of the objective function $J_{M(w)}$: find derivative of $J_{M(w)}$ wrt to w. Let's denote $Y = Xw - b$, then $J_{m(w)} = Y^tY$; now differentiating $J_{M(w)}$ with respect to w:

$$\begin{aligned}\frac{\partial J_{M(w)}}{\partial w} &= \frac{d}{dw}(Y^tY) \\ &= 2Y^t\frac{\partial Y}{\partial w} \\ &= 2(Xw - b)^tX\end{aligned}$$

By setting the gradient to zero, this becomes:

$$2(Xw - b)^tX = 0 \Rightarrow X^tXw = X^tb$$

If we multiply by $[X^tX]^{-1}$, we get $w = [X^tX]^{-1}X^tb = X^{\#}b$ where $X^{\#} = [X^tX]^{-1}X^t$ is the *pseudo-inverse of* X. Consequently, $w = X^{\#}b$ is the optimal solution according to the MSE method. In properties of MSE:

- $w = X^{\#}b$ shows that MSE solution depends on the margin vector b.
- If b is arbitrarily fixed, there is no evidence that the MSE solution gives a separator vector in the case of linear separability.
- MSE pays too much attention to **outlier examples**. Because MSE squares the error, larger error have a disproportionate influence on the solution.
- For classification, where *we only care about which side of the boundary a sample is on*, this sensitivity is undesirable.
- $X^{\#}$ is difficult to obtain, as this requires inverting a large matrix which is expensive for high-dimensional data, impossible if X^tX is singular (non-invertible)

3.6 Least MSE

Considering one **sample at a time** we get the LMS rule:

$$w_{k+1} = w_k + \rho_k(b_i - w_k^tx_k)x_k$$

such that b_k is the value for the k-th sample of the margin b i.e. it depends on the sample x_k. This method is called Widrow-Hoff, least-mean-squares (LMS) or delta rule. It reduces storage requirements by considering single samples sequentially.

Windrow-Hoff rule «corrects» w_k whenever $w_k^tw_k \neq b_k$. The weight change is:

$$\delta w_k = w_{k+1} - w_k = \rho_k(b_k - w_k^tx_k)x_k$$

1. Prediction is correct:
   - $b_k - w_k^tx_k = 0$
   - $\delta w_k = 0$

- No change in the weight vector.

2. Prediction is incorrect:
   - $b_k - w_k^t x_k \neq 0$
   - $\delta w_k \neq 0$
   - Weight vector is updated in the direction of the sample x_k.

3.7 The Ho-Kashyap method

The main limitation of the MSE criterion is the lack of guarantees that a separating hyperplane will be found in the linearly separable case. All we can say about the MSE rule is that it minimises:

$$\|Xw - b\|^2 \text{ when } b \text{ is chosen arbitrarily}$$

This means that whether MSE finds a separating hyperplane or not depends on how properly the margin b is selected (notice that in some sources b is referred to as target outputs).

If the training samples are linearly separable, then there must be vectors w^* and b^* such that $Xw^* = b^* > 0$ where $b* > 0$, meaning that every component of b^* is positive. If b^* were known, one could simply use MSE solution to compute separating hyperplane. The idea is to both find w^* and b^*. Minimize the following criterion function, restricting to positive b:

$$J_{hk}(wb) = \|Xw - b\|^2$$
$$J_{hk}(w^*b^*) = 0$$

As usual take partial derivatives wrt to b and w:

$$\frac{\partial J_{hk}(wb)}{\partial w} = 2X^t(Xw - b)$$
$$\frac{\partial J_{hk}(wb)}{\partial b} = -2(Xw - b)$$

Use gradient descent to find the minimum of $J_{hk}(wb)$: Alternate the two steps below until convergence.

- Fix b and minimise $J_{hk}(wb)$ wrt w
- Fix w and minimise $J_{hk}(wb)$ wrt b

Let $e_k = Xw_k - b_k$ in the linearly separable case:

- If $e_k =$ the solution has been found.
- One of the components of e_k is positive, algorithm continues

In non separable case:

- e_k will have only negative components eventually, thus found proof of non-seperability
- No bound on how mainy iterations is needed for the proof of nonseperability.